

Week 5 — Testing & Quality: Test Report

Eungchan Kang · SWENG 861 Software Construction · Repository `psu-edu/sweng861-crud-ekk5635`, folder `week-05-testing/` · main at `3686338` (2026-09-23)

1. Where the tests live

The Week 4 backend (FastAPI, PostgreSQL) and frontend (Vue 3, Vite) were copied into `week-05-testing/` and the suites were extended there, so the graded Week 4 folder is untouched. Backend tests are in `backend/tests/`: nine files, 96 tests. `conftest.py` populates the environment before anything imports `config`, replaces every credential with a fake, signs test tokens with a test key, and decides from `TEST_DATABASE_URL` whether the database-backed tests run or skip. Frontend tests are in `frontend/src/__tests__`: twelve spec files, 103 tests, run by Vitest with Vue Test Utils, plus one integration spec that drives the real client against the running stack and skips without a session token.

2. Backend unit tests

The 19 tests in `test_oidc.py` (12) and `test_users.py` (7) run with no database and no network, in Arrange-Act-Assert form. Before this week those modules were at 43% and 38% because the callback tests replaced them with lambdas, so none of their rules were checked by anything. Google's two HTTP calls are faked with `monkeypatch`, and the `id_token` is signed with an RSA key generated in the test so that `jwt.decode` runs for real. The cases: discovery reads the four endpoints and is fetched once per process; a failed token exchange or a response without an `id_token` becomes `OIDCError`; a token signed by anyone but Google, minted for another client, or replayed from another login attempt (nonce mismatch) is refused; an unverified e-mail is dropped and the login continues. `upsert_user` is driven by a stub `Session` implementing only the four methods it calls: a first login creates a row keyed by the Google subject, a returning login reuses it, the lookup is by subject and never by e-mail (asserted on the statement's `where` clause), a changed profile is written back, and `last_login_at` is stamped even when nothing else moved. Six single-line mutations of the production code (nonce, audience, `email_verified`, lookup column, the stamp, the reuse branch) each failed exactly one test.

3. Backend integration tests

The remaining 77 tests go through FastAPI's `TestClient`; 47 are marked `needs_db` and run against a test database named by `TEST_DATABASE_URL`, deliberately a different database from the application's, which `conftest.py` creates if it is missing. This was verified: the demo rows behind the Week 4 screenshots survive a full run. The CRUD tests cover create (201 with the stored row and never the `owner_id`), read, list scoped to the caller, patch of only the fields sent, delete (204 and the row is gone), a 409 on a duplicate filer, and the refusals in section 5. This week each refusal asserts the body as well as the status, because a status alone still passes when the body is empty or names the wrong field: a 422 must name `body.title`, a 401 must carry the API's error shape, and a cross-tenant 404 must be byte-for-byte the body an unknown id gets. A POST-then-GET round trip pins the read contract; excluding `status` from the read model fails that test and nothing else.

4. Frontend tests

The 103 tests are component and module tests. The assignment allows component tests, E2E, or a mix; adding Playwright would have cost a new runner and browser install for flows the component tests already exercise. Pure modules are tested without mounting anything: form validation (15), the session (13, including that the token never reaches web storage or the console), the API client (15, including 401 clearing the session and 403 keeping it), `useRequest` (6) and the flash notice (4). The views are mounted with a real router

and a mocked API: the login page reads the token from the URL fragment and strips it; the list shows loading, empty, error-with-retry and populated states; the detail page distinguishes 404, 403 and other failures; the form validates locally, maps a 422 onto its fields, shows the 409 message, disables Save while a request runs and sends exactly one request on a double submit. Two specs render hostile input and assert no element is created. The loading, error and redirect cases the assignment names were already covered from Week 4, which re-measuring confirmed before any test was written; the one real gap, the stale-result guard on `useRequest`'s failure path, received one test, and deleting the guard fails only that test.

5. Auth and authorization scenarios

Scenario	Backend test	Frontend test
401, no or bad token	<code>test_unauthenticated_request_is_rejected</code> , <code>test_bad_tokens_are_rejected</code>	<code>client.spec.ts</code> "signs out and throws on 401"; <code>router.spec.ts</code> "sends a signed-out user ... to /login"
200, own data	<code>test_read_returns_the_owners_row</code> , <code>test_list_returns_only_the_callers_rows</code>	<code>CoveragesView.spec.ts</code> "lists each coverage ..."; <code>CoverageDetailView.spec.ts</code> "... shows the coverage"
404, another tenant's row	<code>test_a_stranger_cannot_reach_another_tenants_row</code> (GET, PATCH, DELETE), <code>test_a_refused_delete_leaves_the_row_alone</code>	<code>CoverageDetailView.spec.ts</code> "tells the user when the item does not exist"
403, admin route	<code>test_an_ordinary_user_is_refused</code> , <code>test_the_refusal_is_403_and_not_the_404_a_tenant_gets</code>	<code>client.spec.ts</code> "keeps the session and throws on 403"; <code>CoverageDetailView.spec.ts</code> "... not theirs to see"

A cross-tenant request answers 404 rather than 403 because a 403 would confirm that the row exists, and an id that answers differently from its neighbours is an id an attacker can enumerate; the test compares the stranger's response with an unknown id's and pins the shared body. The admin route answers 403 because it is one fixed, published path that hides nothing. `owner_id` comes from the token on every request; a caller-supplied `owner_id` is rejected, and no response ever includes it.

6. Coverage summary

Side	Command	Result
Backend	<code>pytest --cov=.</code>	96 passed; 94.38% line coverage over the service (tests, migrations and demo scripts omitted in <code>.coveragerc</code>); gate <code>fail_under = 80</code> . <code>oidc.py</code> and <code>users.py</code> now 100%.
Frontend	<code>npm run test:coverage</code>	103 passed; statements 99.14%, branches 97.67%, functions 98.5%, lines 99.51%; 80% thresholds on all four in <code>vitest.config.ts</code> .

Without `TEST_DATABASE_URL` the backend reads 49 passed, 47 skipped, 76%, and the gate fails. That is intended: a skipped test is not a passed one. Screenshots of both runs and both HTML reports are in `week-05-testing/docs/screenshots/`. No CI workflow exists yet; the README gives the one command per side, and the pipeline is the Week 6 deliverable.

7. Biggest gaps I am aware of

- `db.py` (62%) and `edgar.py` (76%): the engine wiring and the external EDGAR failure paths are exercised only by hand.
- The `api` fixture gives both halves of the POST-then-GET round trip the same session, so it proves the read contract, not persistence across sessions.
- Two frontend lines: the description textarea's input path in `CoverageFormView.vue` and the `ticker ?? '-'` fallback in `CoverageDetailView.vue`.
- Pinned as they are rather than fixed: a failed discovery fetch surfaces as a 500 instead of an `OidcError`; an unverified address clears a verified one already stored; `verify_id_token` allows no clock leeway.
- No browser-level E2E tests, and no Python linter on the backend (the frontend runs `oxlint` and `eslint`).

8. Challenges

- **The unit-test gap was hidden by mocks.** The callback tests swapped `oidc.py` and `users.py` out wholesale, so a green suite said nothing about them. The fix was to fake at the boundary (HTTP, the session) instead of the module.
- **An assertion that could not fail.** The cross-tenant test compared two bodies for equality; two empty bodies would have passed. The shared body is now pinned explicitly.
- **The issue's premise was wrong.** Issue #64 asked for tests that already existed. Re-measuring first avoided the duplicate tests the assignment forbids.
- **A database-less run that looks green.** Because `load_dotenv()` reads `.env`, unsetting `TEST_DATABASE_URL` is not enough; it must be set to an empty string. The coverage gate now makes such a run fail visibly.

9. AI use

- **Where AI helped:** Claude Code drafted the `test_oidc.py` fixtures that generate an RSA key and fake Google's discovery and token endpoints, so that `test_a_replayed_token_from_another_login_attempt_is_refused` runs the real signature check against a mismatched nonce. I reviewed the draft, ran each mutation by hand, and kept it.
- **Where I disagreed and changed it:** for issue #64 the draft proposed three frontend tests for the loading indicator, the error message and the unauthenticated redirect. Re-measuring showed all three already existed with the exact text pinned, so I rejected the draft and wrote one test for the only uncovered branch, the stale-result guard in `useRequest`'s catch path.